Práctica clasificación VCA

Adrián Tinaquero Vizoso

Noel Pan Brea

Grado en Inteligencia Artificial

Manual de uso

- ## Ejecución por línea de comandos:

Debido a la necesidad de evaluar los modelos de forma rápida y cómoda, se ha implementado una script que permite ejecutar por línea de comandos una prueba sobre cualquiera de las redes almacenadas en la carpeta checkpoints, pudiendo elegir el dataset con el que se hará dicha evaluación (En el caso de introducir None en vez de la ruta del csv que contiene el dataset, se utilizará el test_loader del dataset en el que fue entrenado el modelo).

Comando:

```
python execute_test.py modelpath datasetpath
```

Esquema global

- ## Carga y normalización de los datos:

Para esta práctica disponemos de un conjunto de 294 imágenes que corresponden a fotografías de cámaras CCTV pertenecientes a instalaciones portuarias. Con el objetivo de extraer datos para el entrenamiento de los modelos también se proveen dos documentos tipo .csv, ship.csv y docked.csv, los cuales relacionan las imágenes con sus etiquetas ship/no ship y docked/undocked respectivamente.

La carga de datos se realiza mediante un dataset personalizado cuyo código se halla en el archivo dataset.py. Mas tarde en la función load_dataset del módulo data_loader se crean dos datasets, uno con data augmentation (transformaciones en el archivo data_loader.py) y el otro solo con normalización, (mean =0.5 std = 0.225 parámetros estándar de los pesos predeterminados de VGG16) reescalado (224x244 Cabeza default de VGG16) entre otras (transformaciones de formato exclusivamente).

Además, también se dividen los dataset en dataloaders de entrenamiento, validación y test.

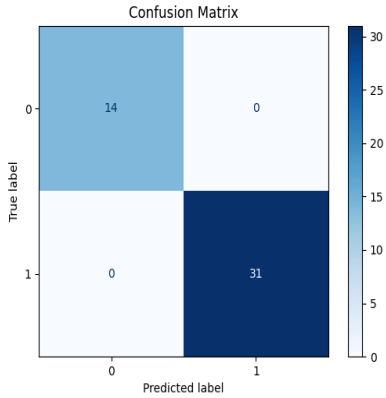
- ## Configuraciones y Resultados:

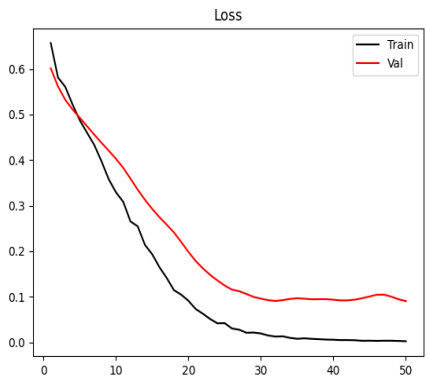
Las diferentes configuraciones utilizadas en esta práctica comparten ciertos parámetros:

1. Lr: El learning rate se ha establecido a 1e-4 dado que tras numeras pruebas se detectó que un valor más alto desestabilizaba en exceso el entrenamiento
2. Arquitectura: Se utilizó la arquitectura VGG16 junto con un clasificador adicional después de las últimas capas (En las configuraciones en las que se utilizan los pesos por defecto todas las capas de vgg salvo las 4 últimas se congelan para evitar incrementar el tiempo de ejecución)
3. Batchsize: Se utilizó un batchsize de 128 ya que es el máximo valor que podía manejar la GPU (Técnicamente era 137 sin embargo decidimos dejarlo en 128 por una cuestión de estándar)

Se han utilizado las siguientes 8 configuraciones en la práctica:

1. PREENTRENADA Y SIN AUGMENTATION: 50 epochs de entrenamiento

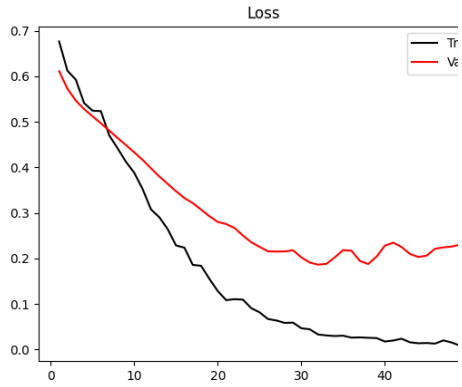


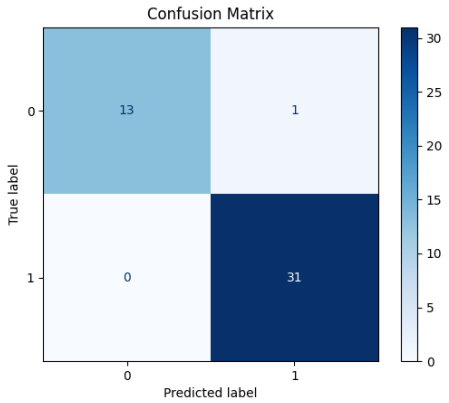



```
Using device: cuda
Usage: python execute_test.py modelpath datasetpath
Using default paths modelpath: VCA/PRÁCTICA 1/checkpoints/PREENTRENADA_SIN_AUGMENTATION.pth, datasetpath: None
Accuracy global: 1.0000

Sensibilidad y Especificidad por clase:
Clase 0:        Sensibilidad=1.0000     Especificidad=1.0000    Precisión=1.0000        VPN=1.0000      F1=1.0
Clase 1:        Sensibilidad=1.0000     Especificidad=1.0000    Precisión=1.0000        VPN=1.0000      F1=1.0
```

2. PREENTRENADA Y CON AUGMENTATION: 50 epochs de entrenamiento

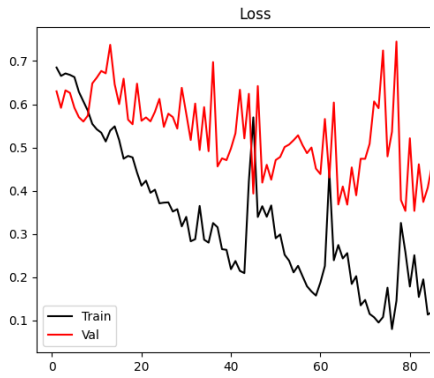
```
Using device: cuda
Usage: python execute_test.py modelpath datasetpath
Using default paths modelpath: VCA/PRÁCTICA 1/checkpoints/PREENTRENADA_CON_AUGMENTATION.pth, datasetpath: None
Accuracy global: 0.9778

Sensibilidad y Especificidad por clase:
Clase 0:        Sensibilidad=0.9286     Especificidad=1.0000    Precisión=1.0000        VPN=0.9688      F1=0.962962962962963
Clase 1:        Sensibilidad=1.0000     Especificidad=0.9286    Precisión=0.9688        VPN=1.0000      F1=0.9841269841269841
```

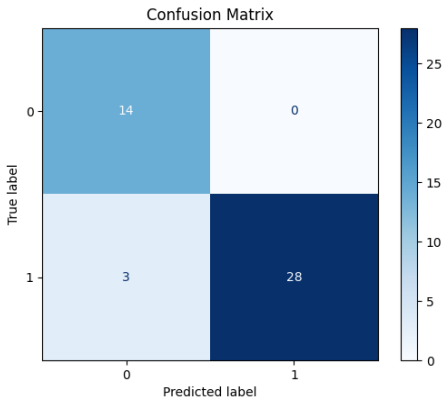




3. NO PREENTRENADA Y SIN AUGMENTATION: 90 epochs de entrenamiento

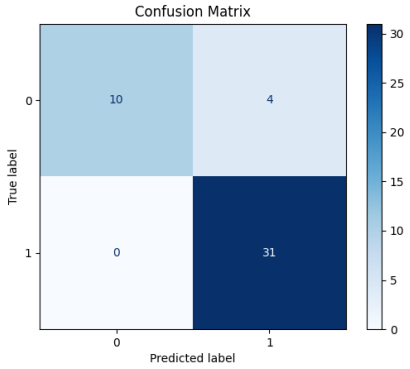
```
Using device: cuda
Usage: python execute_test.py modelpath datasetpath
Using default paths modelpath: VCA/PRÁCTICA 1/checkpoints/NO_PREENTRENADA_SIN_AUGMENTATION.pth, datasetpath: None
Accuracy global: 0.9333

Sensibilidad y Especificidad por clase:
Clase 0:        Sensibilidad=1.0000     Especificidad=0.9032    Precisión=0.8235        VPN=1.0000      F1=0.9032258064516129
Clase 1:        Sensibilidad=0.9032     Especificidad=1.0000    Precisión=1.0000        VPN=0.8235      F1=0.9491525423728813
```





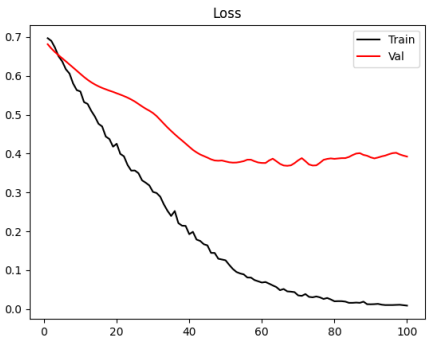
4. NO PREENTRENADA Y CON AUGMENTATION: 90 epochs de entrenamiento

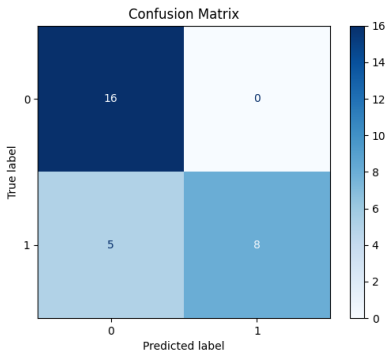
```
Using device: cuda
Usage: python execute_test.py modelpath datasetpath
Using default paths modelpath: VCA/PRÁCTICA 1/checkpoints/NO_PREENTRENADA_CON_AUGMENTATION.pth, datasetpath: None
Accuracy global: 0.9111

Sensibilidad y Especificidad por clase:
Clase 0:        Sensibilidad=0.7143     Especificidad=1.0000    Precisión=1.0000        VPN=0.8857      F1=0.8333333333333333
Clase 1:        Sensibilidad=1.0000     Especificidad=0.7143    Precisión=0.8857        VPN=1.0000      F1=0.9393939393939393
```



5. DOCKED PREENTRENADA Y SIN AUGMENTATION: 100 epochs de entrenamiento

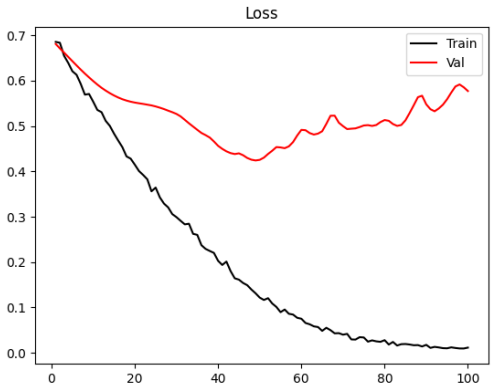
```
Using device: cuda
Usage: python execute_test.py modelpath datasetpath
Using default paths modelpath: VCA/PRÁCTICA 1/checkpoints/DOCKED_PREENTRENADA_SIN_AUGMENTATION.pth, datasetpath: None
Accuracy global: 0.8276

Sensibilidad y Especificidad por clase:
Clase 0:        Sensibilidad=1.0000     Especificidad=0.6154    Precisión=0.7619        VPN=1.0000      F1=0.8648648648648648
Clase 1:        Sensibilidad=0.6154     Especificidad=1.0000    Precisión=1.0000        VPN=0.7619      F1=0.761904761904762
```

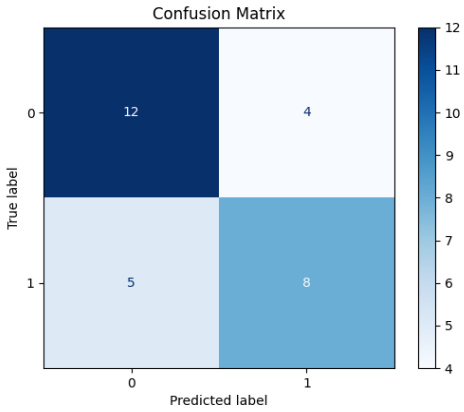




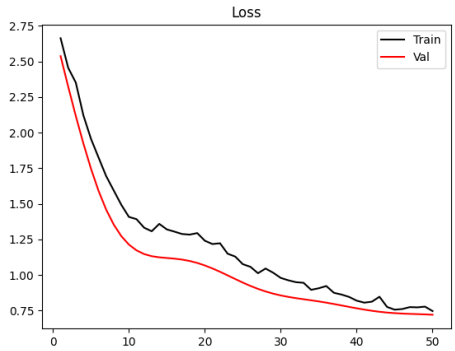
6. DOCKED PREENTRENADA Y CON AUGMENTATION: 100 epochs de entrenamiento

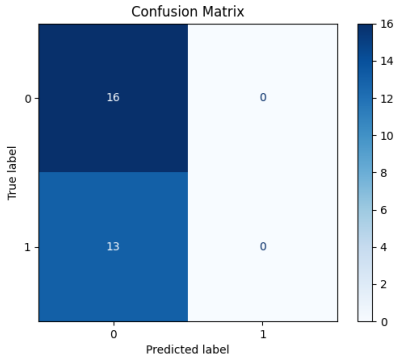
```
Using device: cuda
Usage: python execute_test.py modelpath datasetpath
Using default paths modelpath: VCA/PRÁCTICA 1/checkpoints/DOCKED_PREENTRENADA_CON_AUGMENTATION.pth, datasetpath: None
Accuracy global: 0.6897

Sensibilidad y Especificidad por clase:
Clase 0:        Sensibilidad=0.7500     Especificidad=0.6154    Precisión=0.7059        VPN=0.6667      F1=0.7272727272727272
Clase 1:        Sensibilidad=0.6154     Especificidad=0.7500    Precisión=0.6667        VPN=0.7059      F1=0.64
```

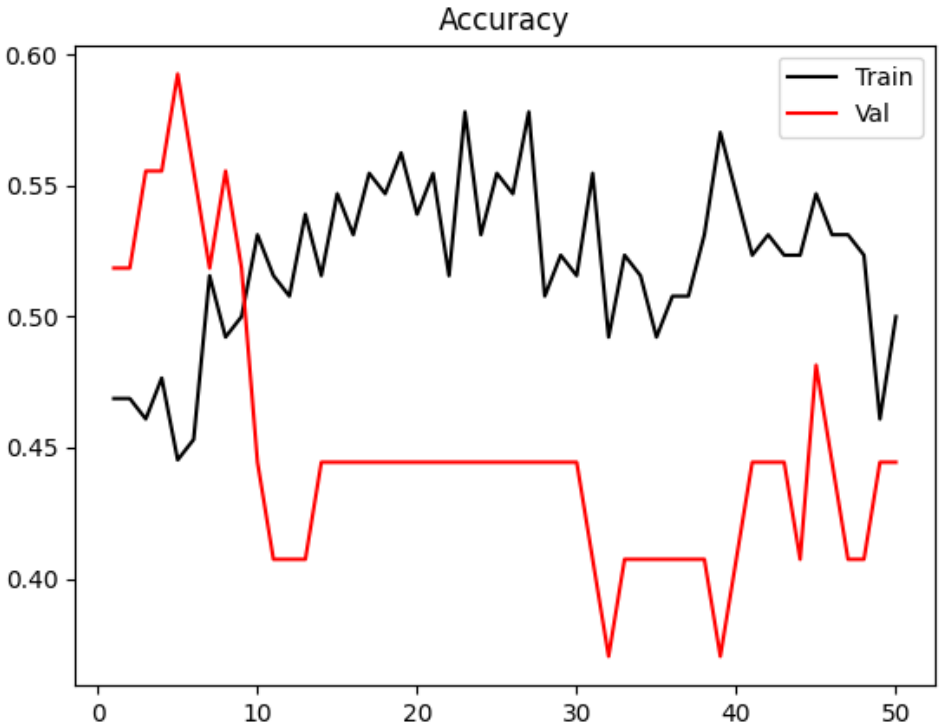




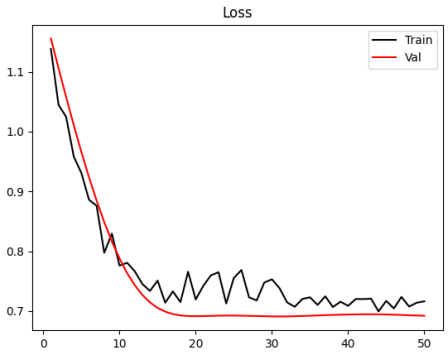
7. DOCKED NO PREENTRENADA Y SIN AUGMENTATION 50 epochs de entrenamiento, utiliza los pesos de las no preentrenadas del ejercicio anterior con fine tunning

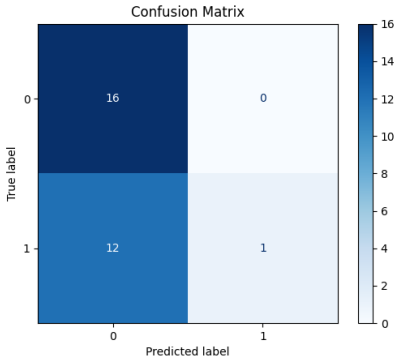


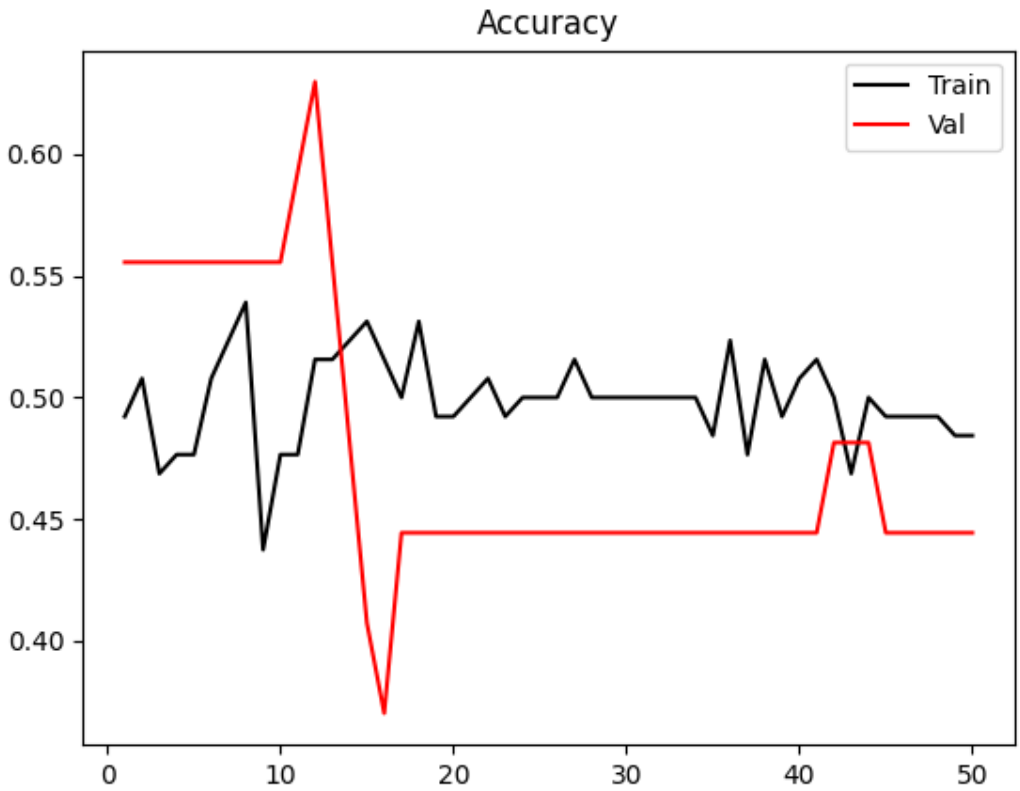




8. DOCKED NO PREENTRENADA Y CON AUGMENTATION 50 epochs de entrenamiento, utiliza los pesos de las no preentrenadas del ejercicio anterior con fine tunning







- Análisis de Resultados:

1. Para el problema Ship/No Ship podemos ver que la red preentrenada sin augmentation es la que mejor rinde obteniendo, prácticamente una puntuación

perfecta (los buenos resultados de las redes en este problema pueden deberse a un posible data leakeage debido a que las imágenes de una misma cámara se utilizan como train y test, aunque esto no afectaría al rendimiento en producción del sistema si no se añaden nuevas cámaras) contrario a la lógica ya que la segunda red que si utiliza augmentation , debería tener un mejor desempeño al disponer de mejores datos, sin embargo esto no es así ya que al no utlizar augmentation en test y al no estar los pesos de la VGG entrenados para tal augmentation el rendimiento disminuye.

Las redes no preentrenadas muestran cierta inestabilidad en el entrenamiento probablemente debido a la gran cantidad de parámetros a ajustar. De todas formas, su desempeño casi alcanza las preentrenadas.

2. Para el problema docked/undocked vemos que las redes preentrenadas superan ampliamente a las no preentrenadas que presentan un colapso y siempre fallan completamente probablemente debido a un error al ejecutar el fine-tunning. De todas formas, el rendimiento no alcanza el del primer problema seguramente debido a la diferencia de complejidad entre ambos